

F24-150-D-Structify

Project Team

Afnan Naseem 21I-1124

Agha Ali 21I-1129

M. Mati Ur Rehman 21I-1222

Session 2021-2025

Supervised by

Mr. Bilal Khalid Dar

Co-Supervised by

Mr. Wasif Ali Wasif



Department of Software Engineering

National University of Computer and Emerging Sciences Islamabad,
Pakistan

October, 2024

Table of Contents

1 Introduction	4
1.1 Problem Statement.....	4
1.2 Problem Solution	4
1.3 Scope.....	5
1.4 Modules	6
1.4.1 Assessment and Diagnostics:	6
1.4.2 Personalized Learning Path:.....	6
1.4.3 Gamified Learning:.....	6
1.4.4 Reward mechanism and Engagement:	6
1.4.5 Real-Time Feedback and Support:	6
1.4.6 Dynamic Code Visualization:	6
1.4.7 User Profile Management:.....	7
1.4.8 Performance Analytics:	7
1.4.9 Settings:.....	7
1.5 User Classes and Characteristics.....	7
2 Project Requirements	8
2.1 Use-case	8
2.1.1 Use Case Diagram	8
2.1.2 High Level Use Cases.....	9
2.1.3 Expanded Use Cases.....	11
2.2 Functional Requirements.....	13
2.2.1 Assessment and Diagnostics:	13
2.2.2 Personalized Learning Path:.....	13
2.2.3 Gamified Learning:.....	14
2.2.4 Reward Mechanism and Engagement:	14
2.2.5 Real-Time Feedback and Support:	14
2.2.6 Dynamic Code Visualization:	14
2.2.7 User Profile Management:.....	14
2.2.8 Performance Analytics:	14
2.2.9 Settings:.....	14
2.3 Non-Functional Requirements.....	15

2.3.1 System Performance:	15
2.3.2 Availability:	15
2.3.3 Scalability:	15
2.4 Domain Model	15
3 System Overview	16
3.1 Architecture Design	16
3.2 Design Models	17
3.2.1 Activity Diagrams:	18
3.2.2 System Sequence Diagrams:	22
3.2.3 Sequence Diagrams:	27
3.2.4 Class Diagram:	32
3.3 Entity Relationship Diagram	34
References	36

Chapter 1

1 Introduction

The project detailed in this document is a **Gamified Intelligent Tutoring System (ITS)** called **Structify**. It is designed to help learners grasp complex data structure concepts like linked lists, stacks, and queues through an interactive and engaging learning platform. The ITS addresses common difficulties students face when learning these subjects, aiming to improve understanding and performance by blending educational content with game-based elements. This project will benefit multiple readers: **team members** involved in building the system, **FYP Panel** evaluating its progress and **supervisor** guiding and evaluating its progress.

1.1 Problem Statement

In computer science education, the course of data structures is fundamental but challenging for students. The concepts of data structures such as linked lists, stacks and queues are abstract which makes it difficult for students to link their theoretical understanding of the concept with the practical application. Due to lack of clarity in these foundational topics students often struggle in this course which results in poor academic results and reduces their ability of succeeding in more advanced courses. Moreover, in conventional teaching, the individual needs of students are not looked after due to which students who need extra attention and require more effort often feel overwhelmed and disengaged.

Studies show that learning data structures is very challenging and less motivating for novice students (Danielsiek et al., 2012; Qian & Lehman, 2017; Silva et al., 2019; Su & Zhang, 2021). Due to the difficulties encountered by students in learning data structures courses, the course is reported to result in high failure and dropout rates in computer science education (García- Mateos & Fernán dez - Alemán, 2009a; Pérez - Sánchez & Morais, 2016). These difficulties in learning data structures should be addressed to improve student performance in this course.

1.2 Problem Solution

To overcome the obstacles encountered in learning data structures, this project proposes a Gamified Intelligent Tutoring System (ITS) that changes the conventional way of learning into an entertaining, custom-made, and interactive way. The system's aim is to make learning fun for students by adopting **gamified elements based on adaptive learning** and **reward mechanisms** while teaching complex data structures principles very efficiently.

The ITS will start with an initial assessment of the user which will be used to create a personalized learning plan for them designed to target their specific needs. The learning plan consists of games based on data structure concepts. When the user is playing a specific game, code of that particular concept is shown along with it and relevant sections are highlighted according to the user's actions. This code visualization will help users understand the underlying code implementation of the data structure concept. Then the user will also be assessed for that concept.

The system will monitor student progress and by utilizing adaptive learning adjusts the learning plan to match the user's needs and pace. The interactive games will have real-time feedback for user's actions to reinforce learning. This combination of interactive games, code visualization, assessments and

personalization will help students master data structure concepts and create a fun and engaging learning environment

1.3 Scope

The Gamified Intelligent Tutoring System (ITS) will teach the concepts of data structure in an interactive and personalized environment. When the user will login to our system, they will have an introductory session which explains the platform and then the system conducts an initial assessment to judge the knowledge level of the user about data structures.

The initial assessment consists of questions from various data structure topics to assess the user's knowledge about each topic and based on this initial assessment the system creates a personalized learning plan for the user. The personalized learning plan will be more focused towards the areas identified in the result of initial assessment check and will begin with foundational concepts and then moves to more advanced and complex topics.

Each main topic, such as **Linked Lists, Stacks, and Queues**, will be divided into sub-topics. For example, Linked Lists will include insertion, deletion, and traversal as sub-topics. Each subtopic will then be broken down into multiple levels, such as insertion at the front, insertion at the back, and insertion at the end. Furthermore, these levels will have three difficulty **stages**: easy, medium, and hard.

For instance, when learning about Linked Lists:

- Insertion will be divided into levels like insertion at different positions (front, back, end).
- Each level will have a progressive learning structure across three stages (easy, medium, hard), ensuring users grasp the concept thoroughly before moving forward.

While learning about each concept, users will interact with gamified lessons that involve realtime code implementation, allowing them to visualize the underlying principles. For example, during the "stack" lessons, users will engage in a game that simulates pushing and popping elements, with accompanying code that dynamically shows how the operations are executed. Tooltips will explain the concepts being applied during the game.

As users move through these levels, the system will monitor their progress and adapt the difficulty of games accordingly. If a user encounters difficulty with a specific stage in a level, they would be offered a relatively easier stage to play. Users who show mastery will be given the opportunity to advance through levels more quickly.

Gamified elements will continue to motivate users. Points and badges will keep users engaged and provide a sense of accomplishment. The system will also offer real-time feedback after each lesson, with explanations of errors to help reinforce learning.

Every concept in the system will end with a final assessment and upon clearing that assessment the user's will earn rewards which will boost their confidence and will motivate them to continue further. Along with rewards, users will earn shareable badges to showcase their achievement.

A user profile will track their progress, achievements, and areas of difficulty over time. Users can return to the platform to revisit previous lessons, refresh their knowledge, or challenge themselves with new

material. The ITS will serve as an ongoing resource to support users throughout their learning journey, from beginner to advanced levels.

Key Topics Covered:

- Linked List
- Stacks
- Queues

1.4 Modules

Our system would be divided into the following modules:

1.4.1 Assessment and Diagnostics:

- This module will be responsible for assessing the initial knowledge level of users through diagnostic tests. It will generate personalized learning plans based on the assessment results and continuously evaluate users' progress throughout the course. This module will also provide final assessments to gauge overall comprehension and readiness for real-world application.

1.4.2 Personalized Learning Path:

- Based on the diagnostics, this module will create and manage customized learning plans for each user. It will organize the learning content in a sequence that suits the user's needs, ensuring that foundational concepts are thoroughly understood before progressing to more advanced topics. The module will adapt the learning path in real-time based on the user's ongoing performance.

1.4.3 Gamified Learning:

- This core module will encompass the various data structure topics, including linked lists, stacks and queues through gamification. Each topic will be broken down into interactive games. The module will ensure that users engage with the material in a meaningful way, bridging the gap between theory and practical application.

1.4.4 Reward mechanism and Engagement:

- This module will incorporate game-like elements such as points and badges and achievement tracking to motivate users. It will manage the rewarding system, track user progress, and display achievements, creating an engaging learning environment.

1.4.5 Real-Time Feedback and Support:

- This module will provide instant feedback to users as they complete exercises and assessments. It will explain errors and guide users through the correct solutions, helping to reinforce learning.

1.4.6 Dynamic Code Visualization:

- This module will generate real-time code representations as users interact with the system's exercises and games. As users manipulate data structures (e.g., adding an element to a stack or traversing a tree), the corresponding lines of code will be dynamically generated and displayed. The specific line of code being executed will be

highlighted, allowing users to directly see the connection between their actions and the underlying code. This feature will enhance understanding by providing immediate, visual feedback on how data structure operations translate into actual code, bridging the gap between theory and practice.

1.4.7 User Profile Management:

- This module will handle all aspects of user management, including account creation, profile management, authentication, and access control. It will ensure that users can securely sign up, log in, and manage their profiles. Additionally, it will track user progress and maintain individual learning histories, allowing users to revisit their learning paths at any time.

1.4.8 Performance Analytics:

- This module will provide analytics based on user performance and engagement. Learners will have access to insights into how they are progressing, which areas are most challenging, and overall trends in learning outcomes. This data will be crucial for continuously improving the system and tailoring content to better meet the needs of users.

1.4.9 Settings:

- This module will allow users to customize the platform's appearance with themes like light or dark mode, adjust font size and style for better readability, and personalize the dashboard layout. They can also set their preferred language for the interface and content. Additionally, the module allows users to submit feedback, helping improve the platform.

These modules will work together to create a cohesive and effective learning environment, ensuring that users can master data structures in an engaging, personalized, and supportive setting.

1.5 User Classes and Characteristics

The primary users of the system will include the following classes:

- **Learner:** Student with a basic understanding of programming fundamentals who want to improve their knowledge of data structures.
- **Administrator:** Who will manage the users and oversee the operations of the system.

Chapter 2

2 Project Requirements

This chapter describes the functional and non-functional requirements of the project. The requirement gathering technique we chose is Use Case.

2.1 Use-case

This section includes the Use Case diagram and high level use cases of the entire project along with the expanded use cases for this iteration

2.1.1 Use Case Diagram

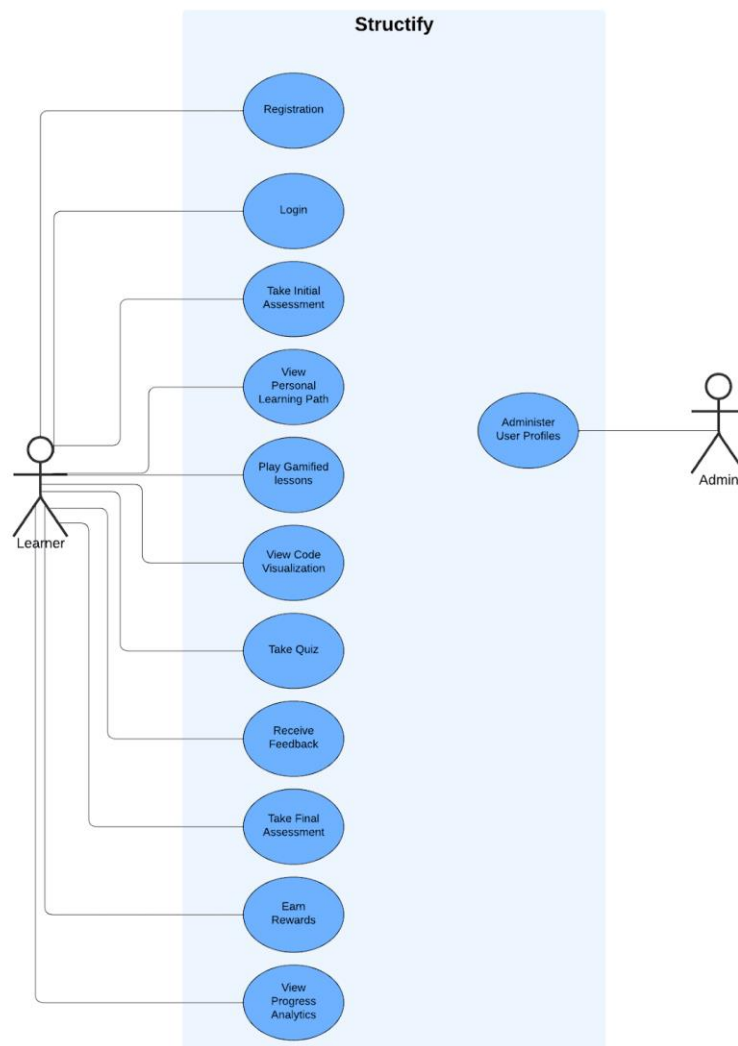


Figure 1: Use Case Diagram

2.1.2 High Level Use Cases

UC-01: Registration	
Use Case	Registration
Actors	Learner
Type	Primary
Description	The learner registers for an account on the platform by providing personal details, such as username, email, and password.

UC-02: Login	
Use Case	Login
Actors	Learner
Type	Primary
Description	The learner logs into the system using their registered username and password to access their personalized learning environment.

UC-03: Take Initial Assessment	
Use Case	Take Initial Assessment
Actors	Learner
Type	Primary
Description	The learner completes an initial test designed to assess their current knowledge of data structures. The results of this test are used to customize their learning plan.

UC-04: View Personalized Plan	
Use Case	View Personalized Plan
Actors	Learner
Type	Secondary
Description	Based on the results from the initial assessment, the system generates and displays a personalized learning path, tailored to the learner's strengths and weaknesses in data structures.

UC-05: Play Gamified Lessons	
Use Case	Play Gamified Lessons
Actors	Learner
Type	Primary
Description	The learner engages with interactive games that cover topics such as linked lists, stacks, and queues. These games are designed to teach data structures in an engaging way.

UC-06: View Code Visualization	
Use Case	View Code Visualization
Actors	Learner

Type	Secondary
Description	As the learner interacts with games, the system dynamically shows real-time code to visualize the underlying data structure operations, such as inserting or deleting nodes.

UC-07: Take Quiz

Use Case	Take Quiz
Actors	Learner
Type	Primary
Description	After completing each level of the gamified lessons, the learner is required to take a quiz to assess their understanding of the topic before progressing further.

UC-08: Receive Feedback

Use Case	Receive Feedback
Actors	Learner
Type	Secondary
Description	The learner receives immediate feedback after a quiz, or assessment. This feedback highlights mistakes and explains the correct approach to reinforce learning.

UC-09: Take Final Assessment

Use Case	Take Final Assessment
Actors	Learner
Type	Primary
Description	After completing all the lessons, the learner takes a comprehensive final assessment to evaluate their understanding of the entire course.

UC-10: Earn Rewards

Use Case	Earn Rewards
Actors	Learner
Type	Secondary
Description	The learner earns rewards such as badges or points based on their performance in quizzes and assessments. These rewards serve to motivate and encourage the learner to continue progressing.

UC-11: View Progress Analytics

Use Case	View Progress Analytics
Actors	Learner
Type	Secondary
Description	The learner can view detailed reports on their progress, including completed modules, quiz scores, and areas that need improvement.

UC-12: Administer User Profiles

Use Case	Administer User Profiles
Actors	Admin
Type	Primary
Description	The admin manages user profiles, including registration details, user progress, and account status. The admin also ensures the system is functioning correctly.

2.1.3 Expanded Use Cases

UC-01: Registration	
Use Case	Registration
Scope	Structify (Gamified Intelligent Tutoring System)
Level	User Goal
Primary Actor	Learner
Stakeholders and interests	• Learner: Wants to create an account on the platform to access the personalized learning features. • Admin: May review the user base and ensure the platform is functioning properly for new registrations. • Third-Party Authentication Providers: Provide alternative sign up methods for users who choose third-party sign-up (e.g., Google, Facebook).
Preconditions	• The learner must have access to the platform. • The system must be able to load the registration page. • The learner is not already registered on the platform.
Postconditions	• The system creates a new account for the learner and logs them into the platform.
Main Success Scenario	1. The learner opens the system and selects the option to create an account. 2. The system loads the registration form. 3. The system prompts the learner to choose between manual registration or third-party registration. 4. If the learner chooses manual registration, they provide required details (username, email, and password). 5. The system validates the details provided. 6. The system checks if the email or username already exists. 7. If the email/username does not exist, the system creates the account. 8. The system logs the learner into the platform. 9. The learner is now registered and can start using the platform.
Extensions	1a. Learner chooses third-party registration (e.g., Google): • The system redirects the learner to the third-party provider for authentication. • The learner completes authentication. • The system receives confirmation and returns the learner to the platform. • If authentication is unsuccessful, proceed to step 3a. 2a. The system detects invalid input (e.g., missing fields, invalid email): • The system displays an error message indicating the specific issue. • The learner re-enters the correct details and retries. 3a. The system detects that the email or username already exists: • The system displays an error message indicating that the account already exists.

	The learner can either log in or choose a different email/username.
--	---

UC-02: Login	
Use Case	Login
Scope	Structify (Gamified Intelligent Tutoring System)
Level	User Goal
Primary Actor	Learner
Stakeholders and interests	Learner: Wants to access their personalized learning environment after successful authentication. Admin: Wants to ensure that only registered learners with valid credentials can access the system. Third-Party Authentication Providers: Provide alternative login methods for users who choose third-party sign-up (e.g., Google, Facebook).
Preconditions	- The learner must already be registered on the platform. - The system must be functioning and able to process login requests.
Postconditions	The system logs the learner in and provides access to the personalized learning environment.
Main Success Scenario	- The learner opens the system and navigates to the login page. - The system loads the login page. - The system prompts the learner to enter their credentials or choose a third-party login option. - The learner enters their username and password: - The system validates the credentials. - If the credentials are correct, the system logs the learner into their account. - The learner gains access to the personalized learning environment.
Extensions	- Learner chooses third-party login: - The system redirects the learner to the third-party provider for authentication. - The learner completes authentication. - The system receives confirmation and returns the learner to the platform. - If authentication is unsuccessful, proceed to step 3a. - The system detects invalid input (e.g., missing fields, invalid email): - The system displays an error message indicating the specific issue. - The learner re-enters the correct details and retries.

UC-03: Play Gamified Lessons	
Use Case	Play Gamified Lessons
Scope	Structify (Gamified Intelligent Tutoring System)
Level	User Goal
Primary Actor	Learner
Stakeholders and interests	Learner: Wants to learn data structure concepts in an interactive and engaging way by playing games.
Preconditions	- The learner must be logged in to the system. - The learner has selected a lesson or topic to play. - The game should be loaded and ready to start.

Postconditions	The learner completes the game, with their performance tracked and feedback provided.
Main Success Scenario	- The learner selects a gamified lesson (e.g., a topic on linked lists, stacks, or queues). - The system loads the game and gives an introduction or instructions on how to play. - The game displays the initial scene with the task or objective. - The learner makes a move to solve a problem or perform an operation (e.g., inserting an element into a linked list). - If the move is correct, the system displays a success message and updates the score. - The learner continues making moves until the game is completed. - The system updates the learner's score and records their progress.
Extensions	1a. Learner makes an incorrect move: - The system displays an error message explaining the mistake. - The system reduces the learner's life count. - The learner is prompted to try again until they either make a correct move or lose all lives. 2a. Learner loses all life counts: - The game ends, and the system provides feedback on what the learner could improve. - The learner may retry the game or choose another lesson. 7a. Learner exits the game early: - The game is paused, and progress is saved for the learner to return later. - The learner can either resume the game or exit the lesson entirely.

2.2 Functional Requirements

2.2.1 Assessment and Diagnostics:

- The system shall provide an initial diagnostic assessment to evaluate users' current knowledge level in data structures.
- The system shall continuously assess users' progress throughout the learning journey, which will help in tailoring the personalized learning plan.
- The system shall offer a final assessment to gauge users' overall comprehension and readiness for real-world application.

2.2.2 Personalized Learning Path:

- The system shall generate a personalized learning plan based on the user's initial self-identification.
- The system shall adapt the content and sequence of learning materials in real-time based on the user's ongoing performance.
- The system shall ensure that users build foundational knowledge before progressing to more complex data structure topics.

2.2.3 Gamified Learning:

- The system shall offer interactive games and hands-on exercises for each data structure topic including linked lists, stacks and queues.
- The system shall ensure that users engage with the material in a meaningful way, using gamification to bridge the gap between theory and practical application.
- The system shall provide a variety of games to cater to different learning styles of individuals.

2.2.4 Reward Mechanism and Engagement:

- The system shall incorporate a reward mechanism that tracks points, badges, and achievements earned by users.
- The system shall motivate users by offering rewards and recognition for completing modules and mastering concepts.

2.2.5 Real-Time Feedback and Support:

- The system shall provide real-time feedback to users during games and assessments, highlighting errors and explaining correct solutions.
- The system shall ensure that users receive immediate feedback to reinforce learning and boost confidence.

2.2.6 Dynamic Code Visualization:

- The system shall dynamically generate and display corresponding lines of code as users interact with exercises and games.
- The system shall highlight the specific line of code being executed, allowing users to see the direct connection between their actions and the underlying code.

2.2.7 User Profile Management:

- The system shall allow users to create an account, log in, and manage their profiles securely.
- The system shall track user progress, including completed modules, earned rewards, and areas of difficulty.
- The system shall enable users to revisit their learning paths and access their learning history at any time.

2.2.8 Performance Analytics:

- The system shall track and display detailed analytics on each learner's progress, including completed modules, time spent on each activity, and areas where they struggle the most.
- The system shall identify and highlight specific areas or topics where users face the most difficulty, providing visual indicators and suggestions for additional practice or resources.

2.2.9 Settings:

- The system shall allow users to switch between available themes, including light and dark modes, to customize the platform's visual appearance.

- The system shall enable users to adjust the font size and style within the platform for improved readability.
- The system shall allow users to customize the layout of the dashboard and learning modules by selecting between compact and expanded views.
- The system shall provide users with the option to select their preferred language for the interface and content.
- The system shall provide a feature for users to submit feedback directly within the module, including suggestions for improvements.

2.3 Non-Functional Requirements

2.3.1 System Performance:

- The system shall handle up to 1,000 concurrent users without any degradation in performance, ensuring that response times for all actions (e.g., loading modules, displaying code, and providing feedback).
- The system shall have a load time of maximum 5 seconds.

2.3.2 Availability:

- The system shall maintain an uptime of 90% during any calendar month, ensuring that users have continuous access to the learning platform with minimal downtime.

2.3.3 Scalability:

- The system shall be able to scale to support a 50% increase in user base within 30 days without requiring significant changes to the underlying infrastructure

2.4 Domain Model

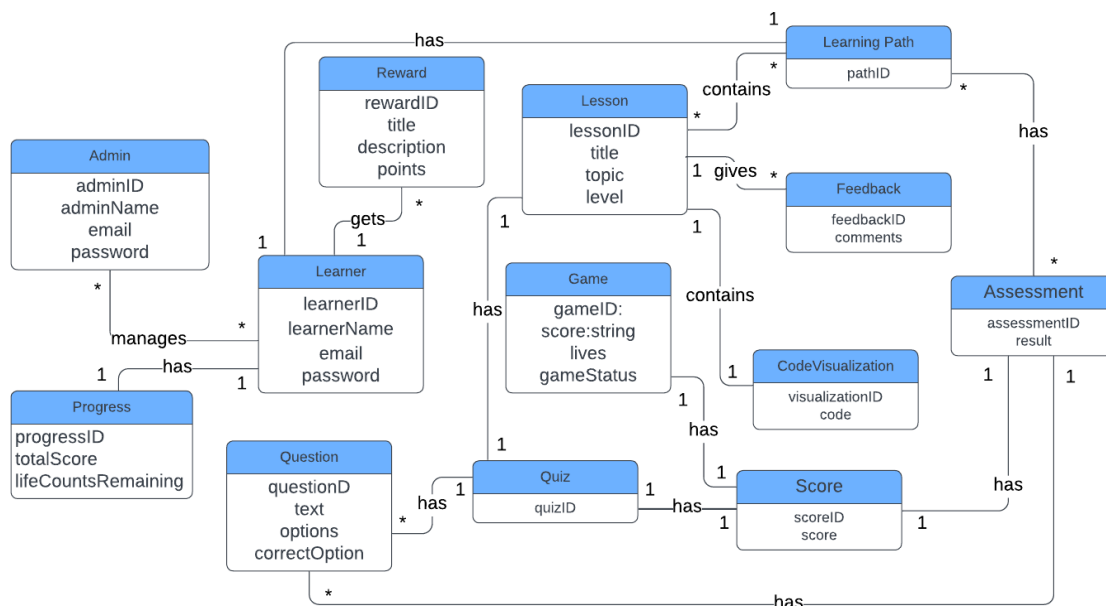


Figure 2: Domain Model

Chapter 3

3 System Overview

This chapter provides a high-level understanding of our system's structure and design. It introduces the key components of the system's architecture, detailing how various layers and modules interact. The overview includes the Architecture Design, outlining the system's layered framework, followed by Design Models, which present activity diagrams, sequence diagrams, and class diagrams to illustrate the system's workflows and interactions. Finally, the Entity Relationship Diagram (ERD) is included to depict the relationships between key data entities within the system. This section serves as a foundational guide for understanding the system's functionality and design.

3.1 Architecture Design

We have composed a layered architecture for our system. The layered architecture diagram for our system is organized into four main layers, each responsible for specific functions:

1. **Presentation Layer:** This topmost layer handles the user interface and user interactions. It consists of components such as:
 - Games: User-facing games as part of the learning experience.
 - Web UI: The user interface that allows interaction with the system.
 - Code Visualization: Provides visual representations of code to aid learning.
 - Progress Visualization: Displays the progress of the learner throughout the system.
2. **Application Layer:** This layer contains the core business logic that governs the system's operations, including:
 - Adaptive Learning: Handles personalized learning paths based on user behavior.
 - Rewards: Manages the system's reward mechanism for learners.
 - Assessments: Administers and processes tests or quizzes for users.
 - Feedback: Collects and provides feedback to learners to aid their improvement.
3. **Data Access Layer:** This layer manages communication between the application logic and the data storage. It contains:
 - MongoDB Queries: Interacts with the database to retrieve or update information.
 - RESTful APIs: Provides an interface for external systems or services to interact with the system's data.
4. **Database Layer:** The bottom layer is responsible for data storage, specifically
 - Database Collections: Houses the system's structured data in collections (relevant to MongoDB).

The architecture follows a clean separation of concerns, ensuring each layer is focused on its specific tasks while communicating with adjacent layers through defined interfaces.

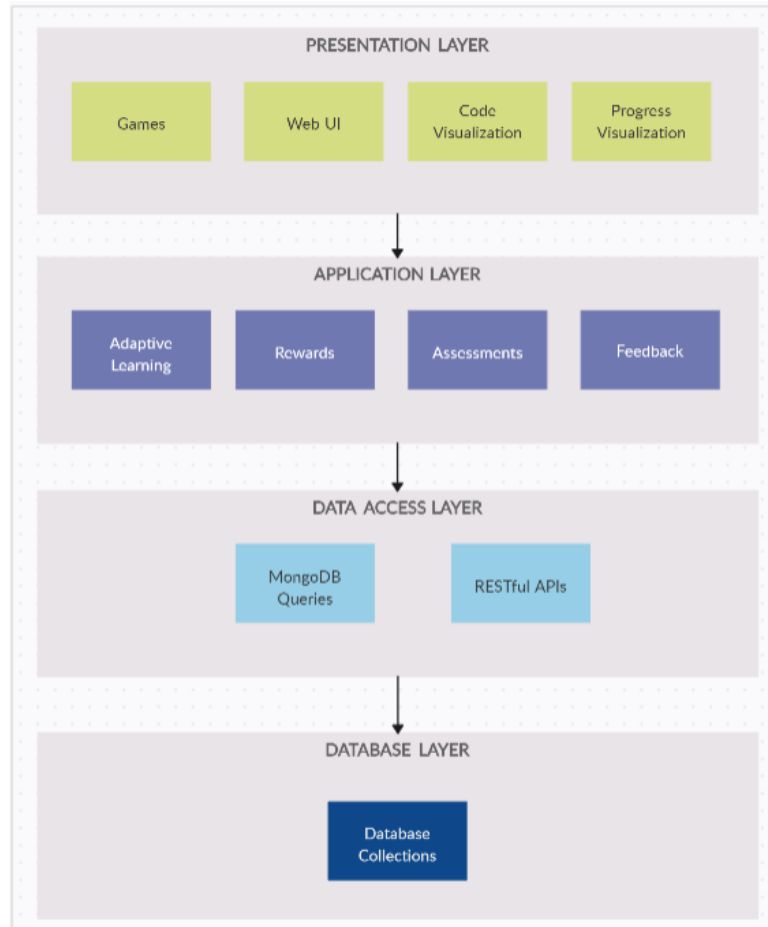


Figure 3: Architecture Diagram

3.2 Design Models

In the design models section, we have used a variety of diagrams to visually represent the system's structure and functionality. These diagrams help in understanding the dynamic and static aspects of the system, illustrating how different components interact with each other.

Activity diagrams are used to capture the flow of activities and actions within the system, highlighting user interactions and system processes. They provide a clear picture of how tasks are accomplished step-by-step.

System Sequence Diagrams (SSDs) focus on the interactions between the user and the system at a high level, showing the sequence of messages exchanged to achieve a specific function.

Sequence Diagrams go deeper into the internal communication between various objects and components within the system, capturing the specific order in which operations and messages are executed to complete a process.

Finally, the **Class Diagram** represents the static structure of the system by showcasing its key entities, their attributes, and relationships. This diagram is essential in illustrating how data is structured and how different components are connected to ensure the system functions cohesively.

These design models together provide a comprehensive view of the system, ensuring both the process flow and structural integrity are well-defined.

3.2.1 Activity Diagrams:

The following activity diagrams represent key user interactions within the system for this iteration, including registration, login, password reset, selecting a game, and playing a game. Each diagram details the flow of actions between the user and the system, outlining how the system processes user input and responds. These diagrams provide a clear understanding of the system's behavior during these core functionalities, ensuring seamless user experiences and effective system operations.

3.2.1.1 User Registration

This diagram outlines the steps involved in user registration. The user begins by opening the system and selecting the option to create an account. They are then given a choice to register either through third-party services or by entering personal details manually. The system loads the relevant signup form, prompts the user for required information, and validates the input. If the details are correct and the account doesn't already exist, the system creates a new account. If the details are incorrect, an error is displayed. Upon successful registration, the user is automatically logged into the system.

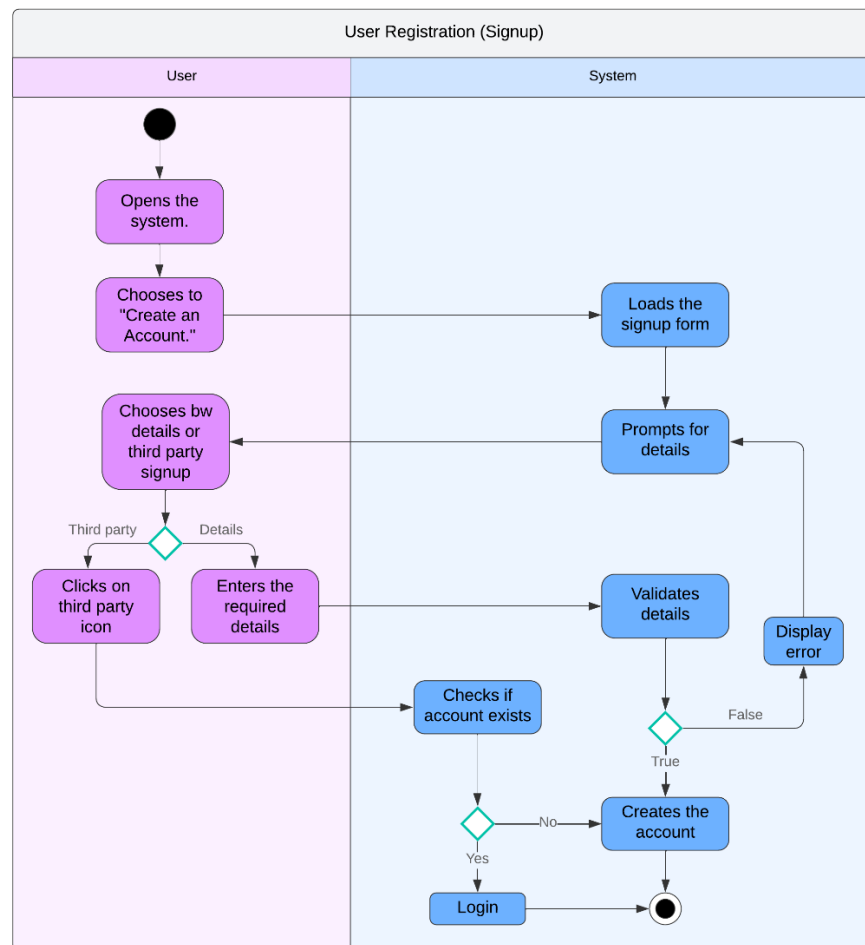


Figure 4: User Registration Activity Diagram

3.2.1.2 User Login

The user login process starts when the user opens the system and navigates to the login page. They can choose between logging in using third-party services or by entering their credentials manually. After the system loads the login page and prompts for credentials, it validates the input. If the credentials are incorrect, an error message is displayed. If they are correct, the system logs the user in, granting access to the system.

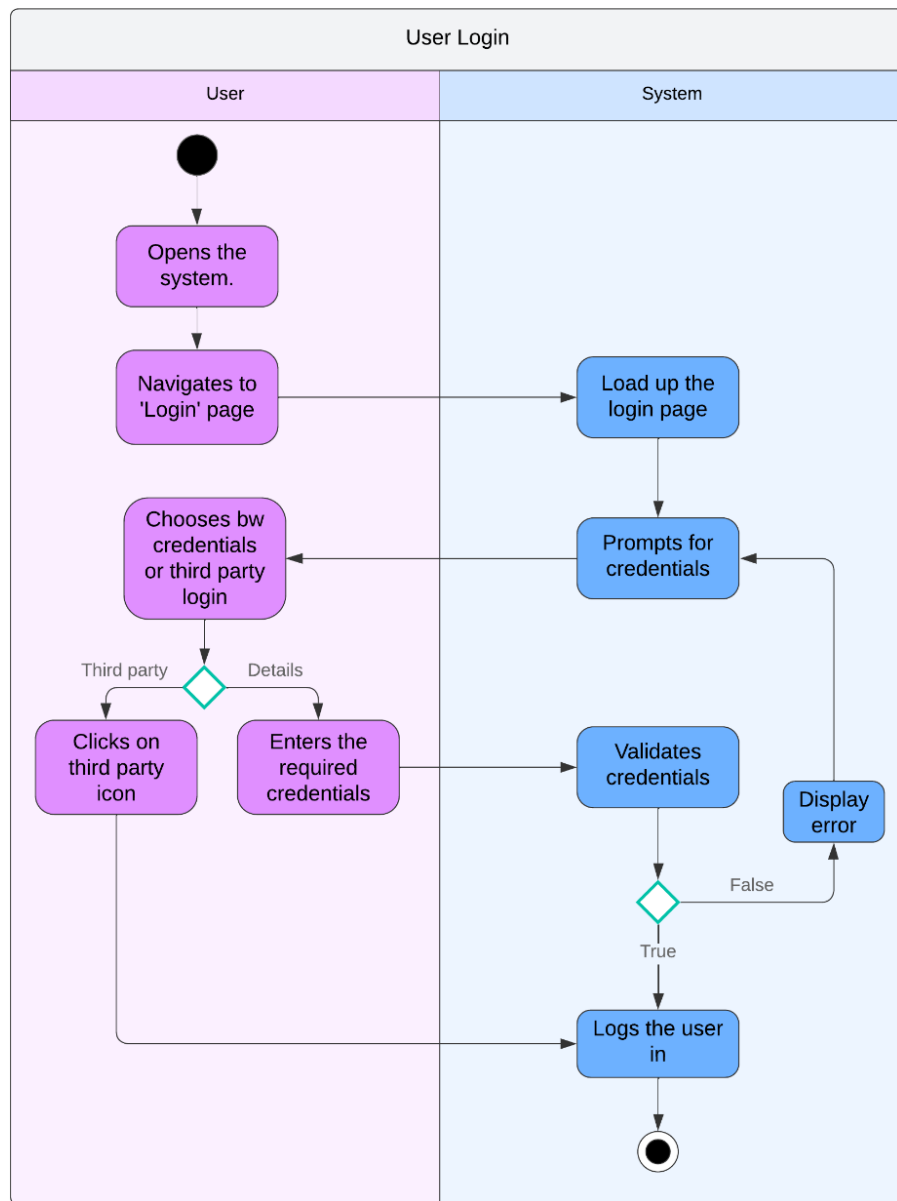


Figure 5: User Login Activity Diagram

3.2.1.3 Reset Password

This activity diagram illustrates the steps involved in resetting a user's password. The process begins when the user navigates to the reset password option in the system. The system then prompts the user to enter their email address. Upon submission, the system validates the email. If valid, a PIN is sent to the user's email; otherwise, an error message is displayed.

The user enters the received PIN, and the system validates the PIN. If the PIN is correct, the system prompts the user to enter a new password. After entering the new password, the system updates the password and completes the process. If the PIN is invalid, an error message is displayed, and the user must retry.

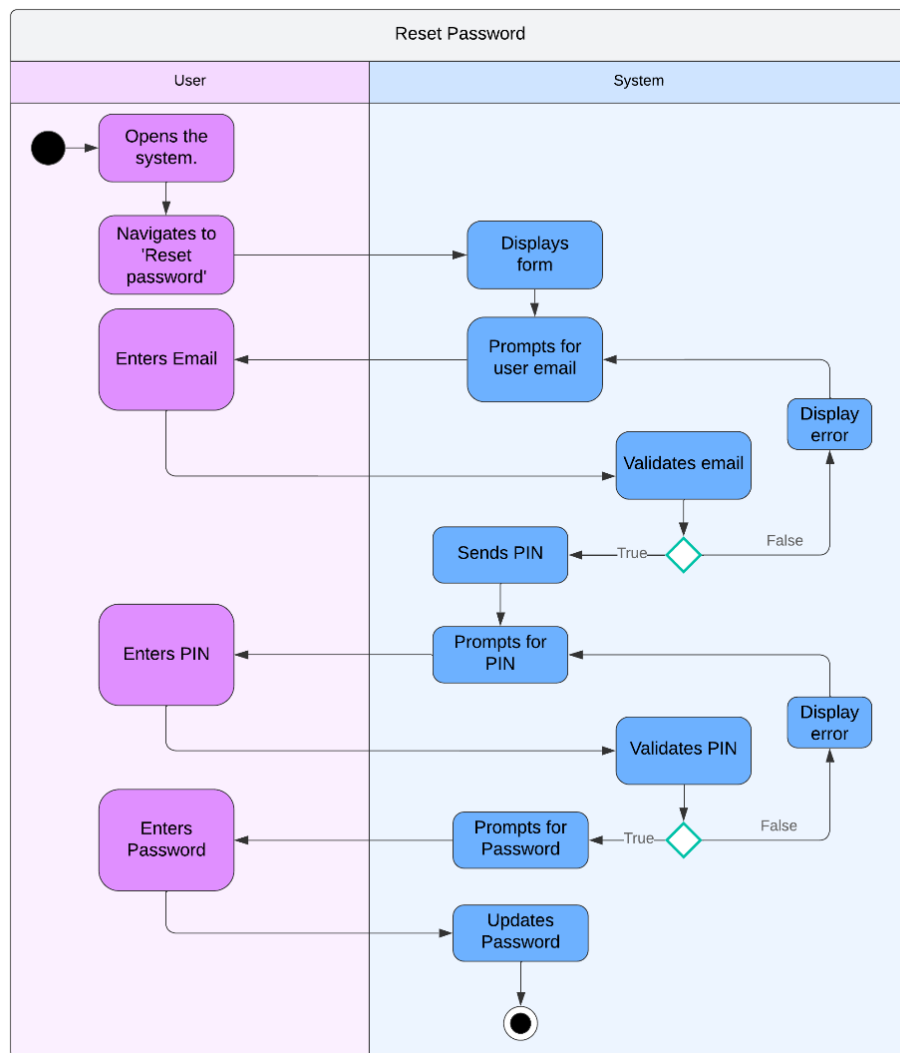


Figure 6: Reset Password Activity Diagram

3.2.1.4 Playing a game:

In this diagram, the user starts by initiating a game. The system provides an introduction and displays the initial scene. The user makes a move, and based on the correctness of the move, the system either displays an error and reduces the number of lives or displays a success message and updates the score. This continues until the game ends or user interaction is complete.

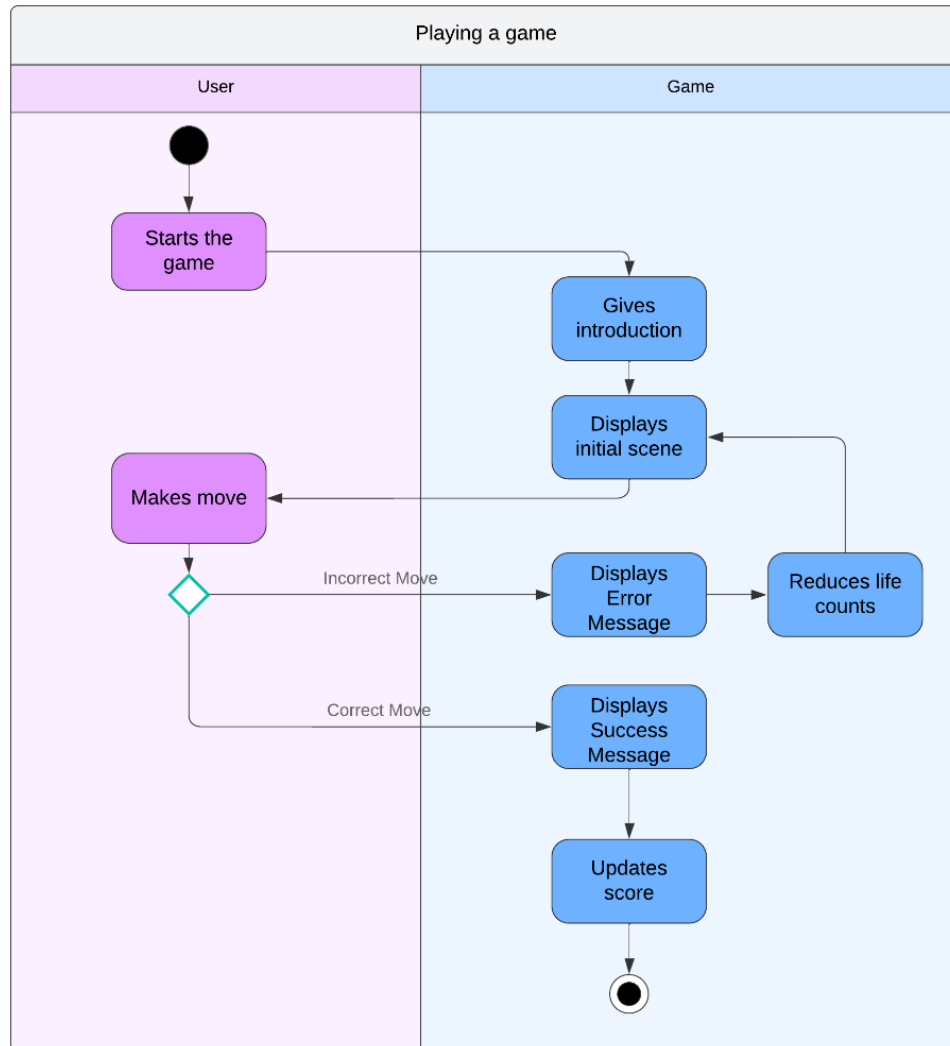


Figure 7: Playing a game

3.2.1.5 Selecting a game

This diagram shows the process of selecting a game within the system. The user logs into the system and navigates to the learning section. From there, they are presented with various data structure topics and select one (e.g., Linked Lists). After selecting a specific sub-topic such as insertion, the system displays the available games and then loads the chosen game.

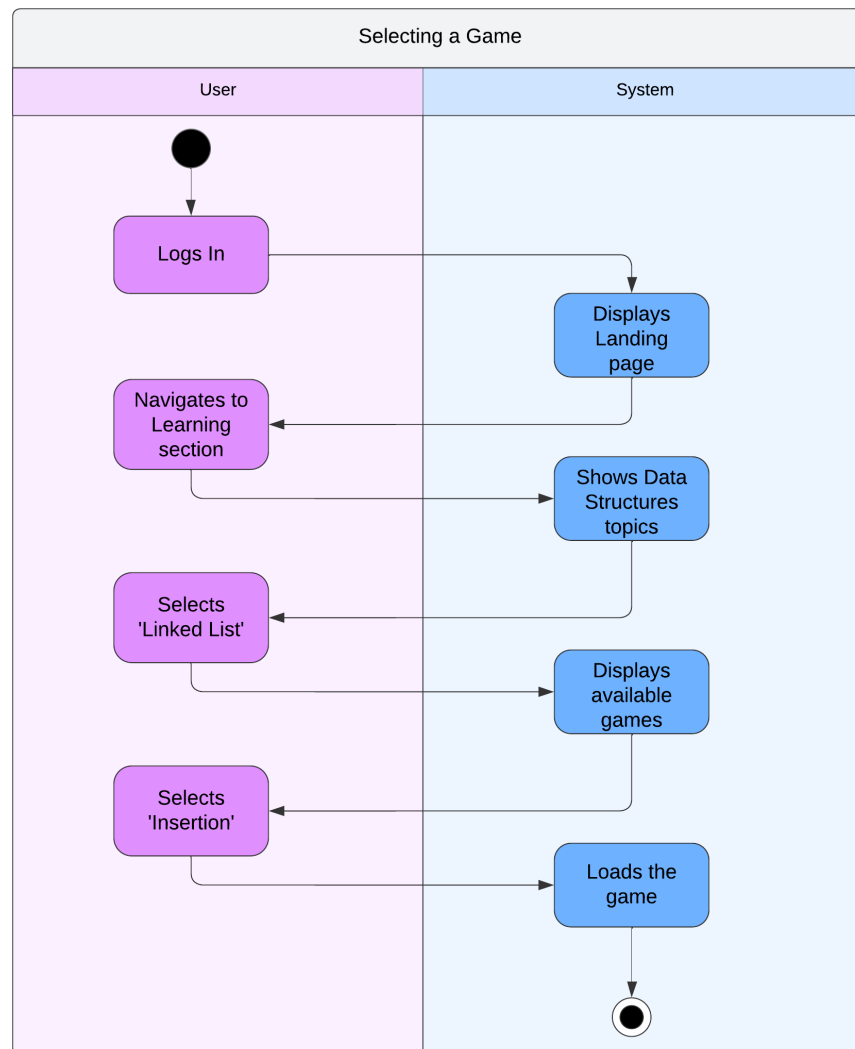


Figure 8: Selecting a game

3.2.2 System Sequence Diagrams:

The following sequence diagrams represent the detailed interaction between users and the system components for key processes in the application. These diagrams focus on the step-by-step message exchanges and workflows involved in actions such as user registration, login, password reset, game selection, and playing a game. Each sequence diagram highlights how the system handles user inputs, validates data, and ensures smooth transitions throughout the user experience.

3.2.2.1 Registration

This diagram represents the user registration process. The user opens the registration form, and the system displays the required fields. The user can either choose to register using third-party services or manually provide details like email, password, and age. The system validates the information and, if correct, creates the user account. A success message is displayed upon successful registration. In case of invalid details, an error message is shown.

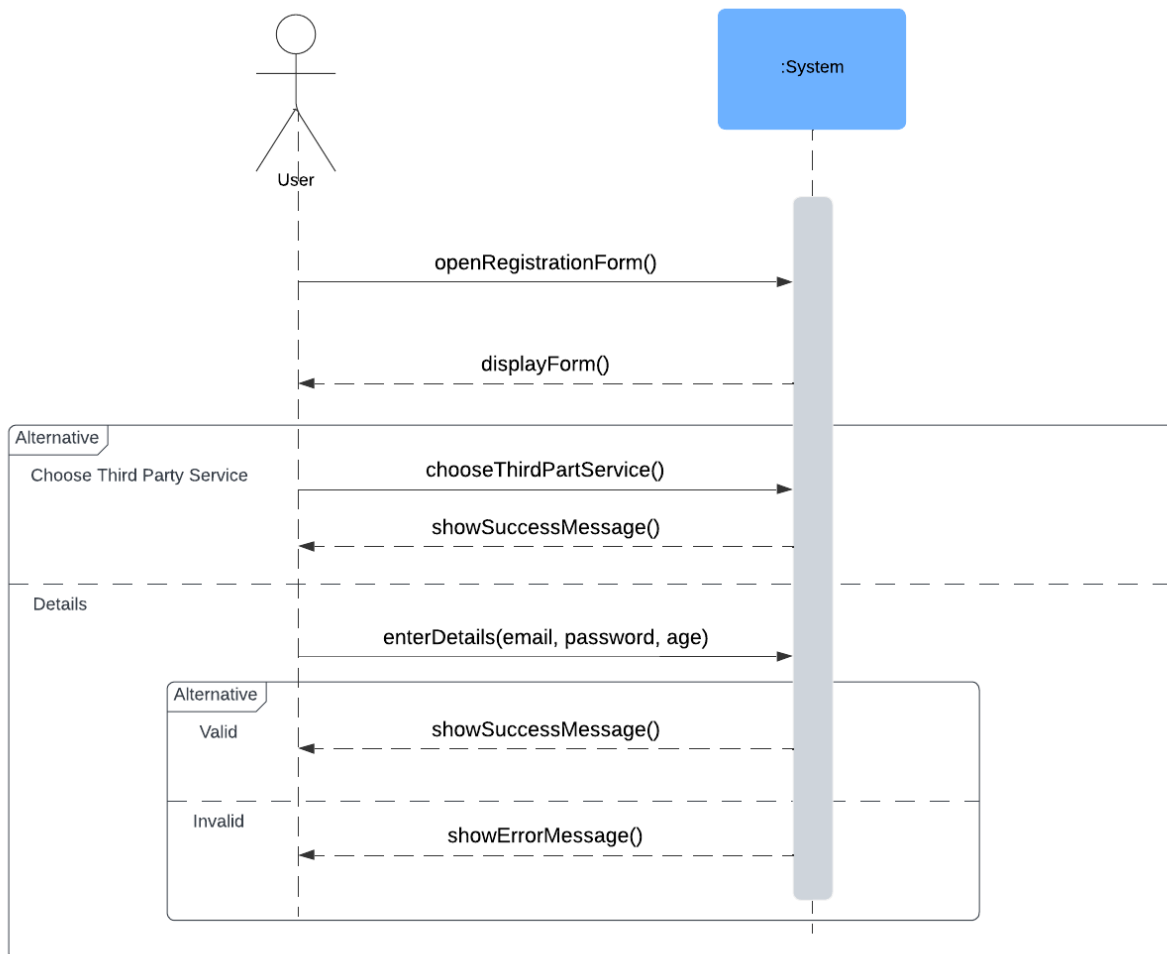


Figure 9: Registration System Sequence Diagram

3.2.2.2 Login

This system sequence diagram illustrates the process of logging into the system. The user begins by opening the login form. The system responds by displaying the form where the user has two options: logging in via third-party services (e.g., Google, Facebook) or manually entering their login credentials. If the user selects a third-party service, the system authenticates and displays a success message. If the user enters their email and password manually, the system validates the credentials. Upon successful validation, a success message is displayed, granting access to the user. If the credentials are invalid, an error message is shown, prompting the user to try again.

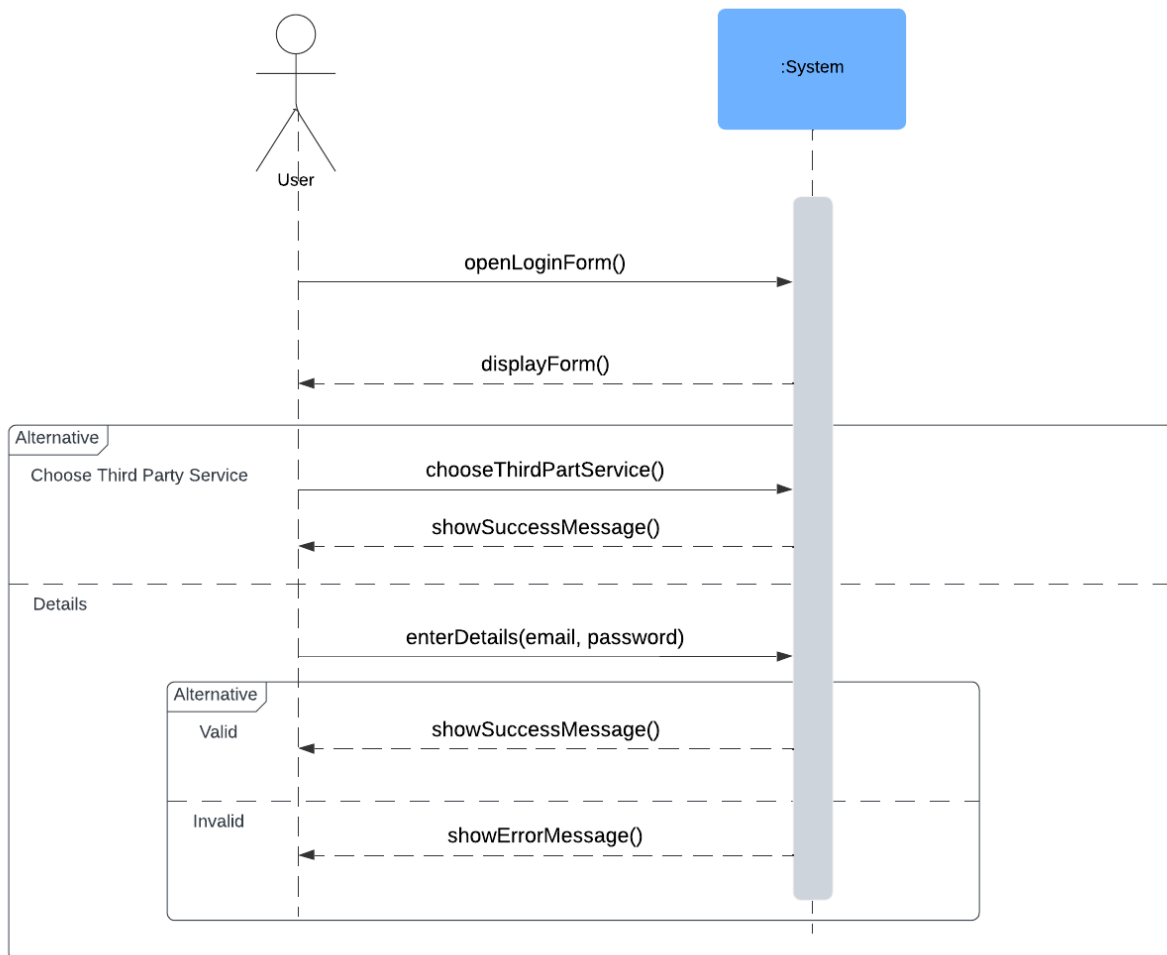


Figure 10: Login System Sequence Diagram

3.2.2.3 Reset Password

This SSD shows the process for resetting a password. The user starts by requesting the reset password form, entering their email, and receiving a prompt to input a PIN. If the PIN is correct, the system allows the user to enter a new password. Upon successful validation, a success message is displayed. If the PIN or new password is invalid, the system shows an error message.

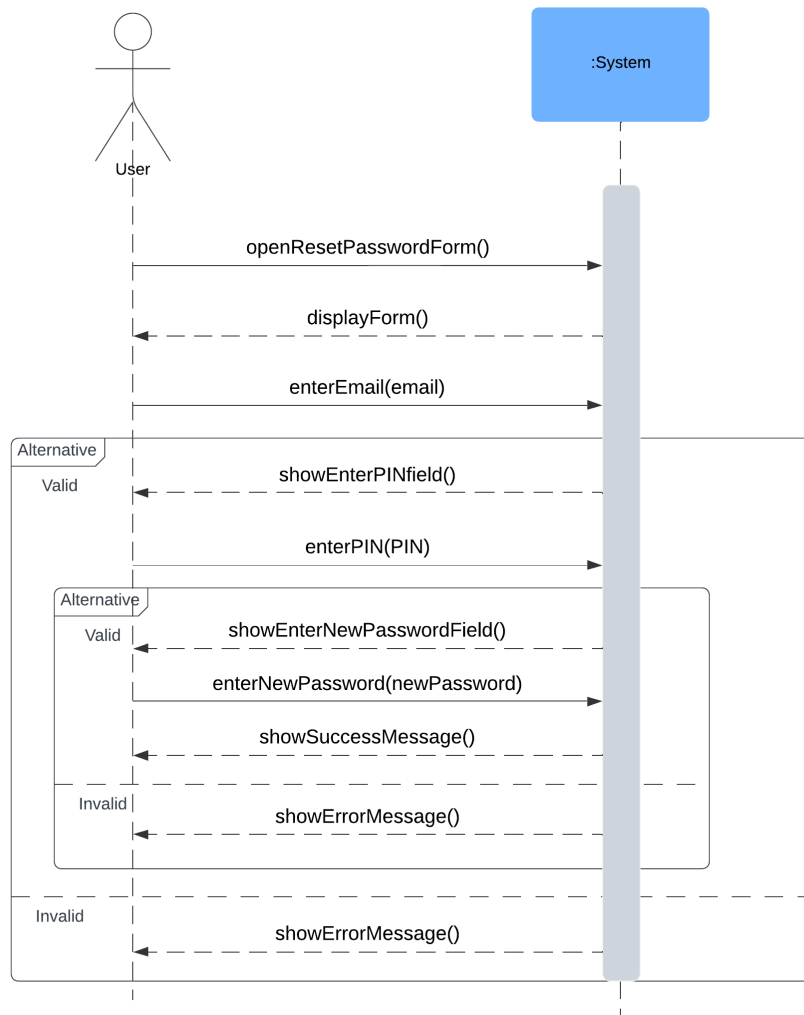


Figure 11: Reset Password System Sequence Diagram

3.2.2.4 Select a Game

This diagram illustrates the process of selecting a game in the system. The user begins by selecting the learning section. The system responds by retrieving and displaying the available topics. Once the user selects a specific game from the topics, the system loads the game environment. This interaction ensures the user can access the learning content in a structured and straightforward manner.

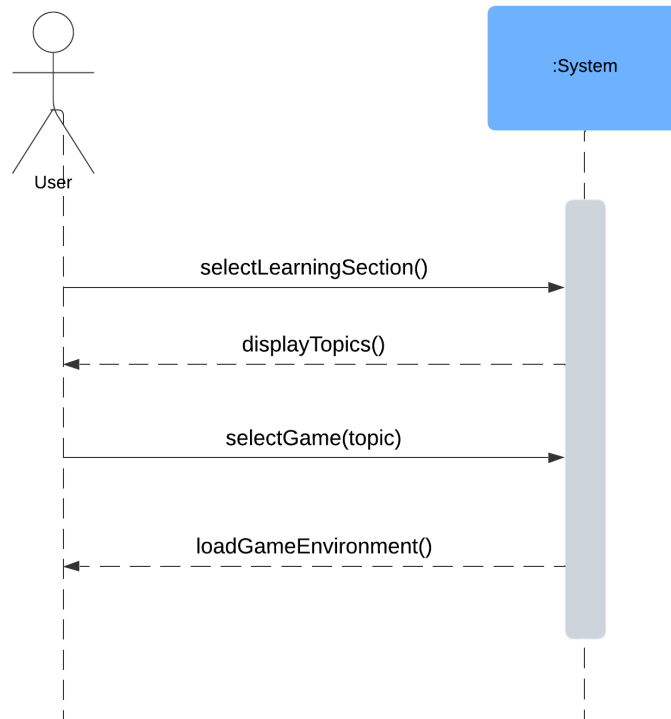


Figure 12: Select a game System Sequence Diagram

3.2.2.5 Play a Game

In the sequence diagram for playing a game, the user starts the game, and the system responds by displaying the game instructions. The user then makes a move, and the system checks if the move is valid or invalid. Depending on the validity of the move, a success or error message is shown. This loop continues until the game is finished. Once the game ends, the system saves the user's progress.

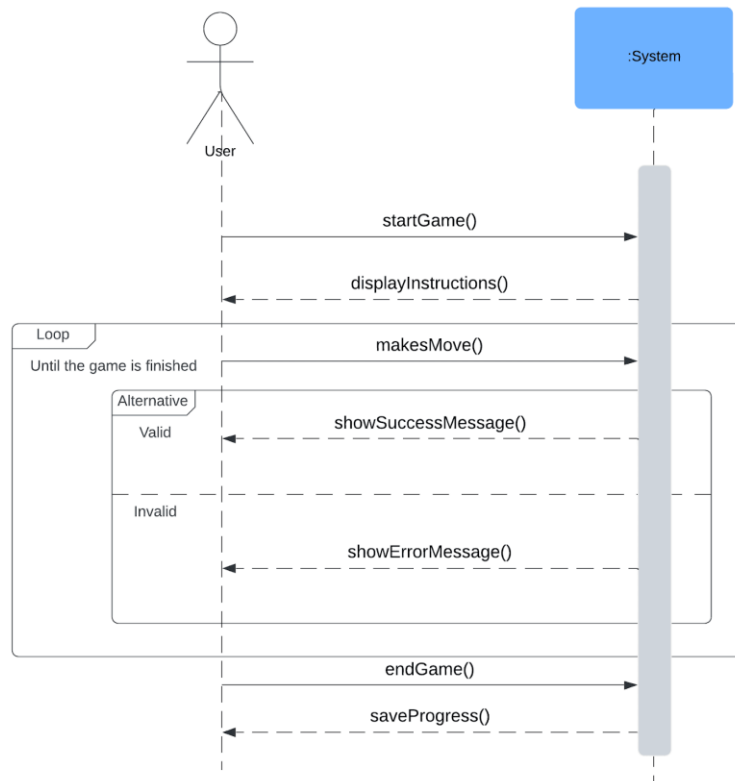


Figure 13: Play a game System Sequence Diagram

3.2.3 Sequence Diagrams:

The system sequence diagrams (SSDs) provide a high-level view of the interaction between the user and the system during essential operations, including registration, login, password reset, and game selection. These diagrams focus on the external inputs from the user and the system's response, illustrating how the system processes user requests and returns the appropriate outcomes. The SSDs help outline the core functions and ensure that system behavior aligns with user actions.

3.2.3.1 Registration

This diagram describes the registration process. The user requests the registration form, and the system responds by displaying the form. The user can choose to register using a third-party service or by entering details manually (email, password, age, and gender). The system validates the information and checks for any duplicates in the database. If the details are valid and unique, the system registers the user and displays a success message. If there are errors, such as an existing account with the same email, an error message is shown to the user.

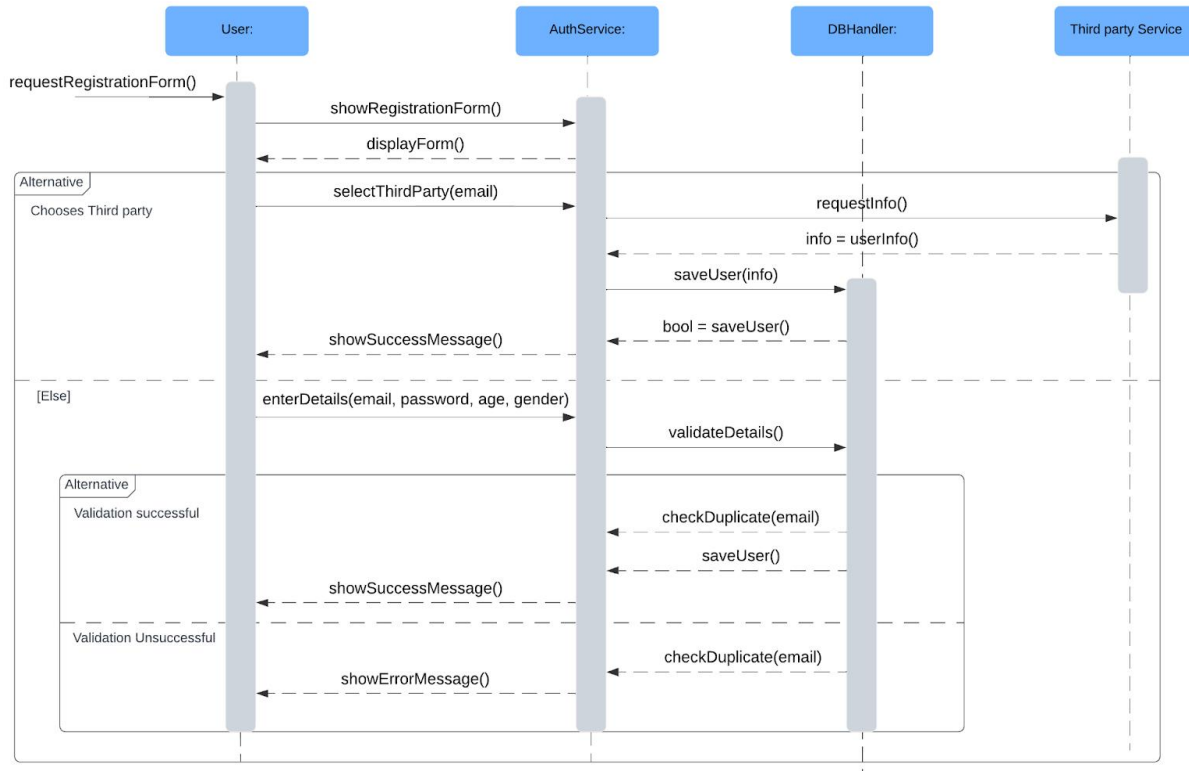


Figure 14: Registration Sequence Diagram

3.2.3.2 Login

The login process begins when the user opens the login form. The system responds by displaying the form. The user then selects whether to log in using a third-party service or by entering their credentials manually (email and password). If the third-party service is chosen, the system performs authentication and shows a success message upon successful login. If the user enters their credentials manually, the system validates the details. A success message is shown upon successful validation, while an error message is displayed if the login attempt fails.

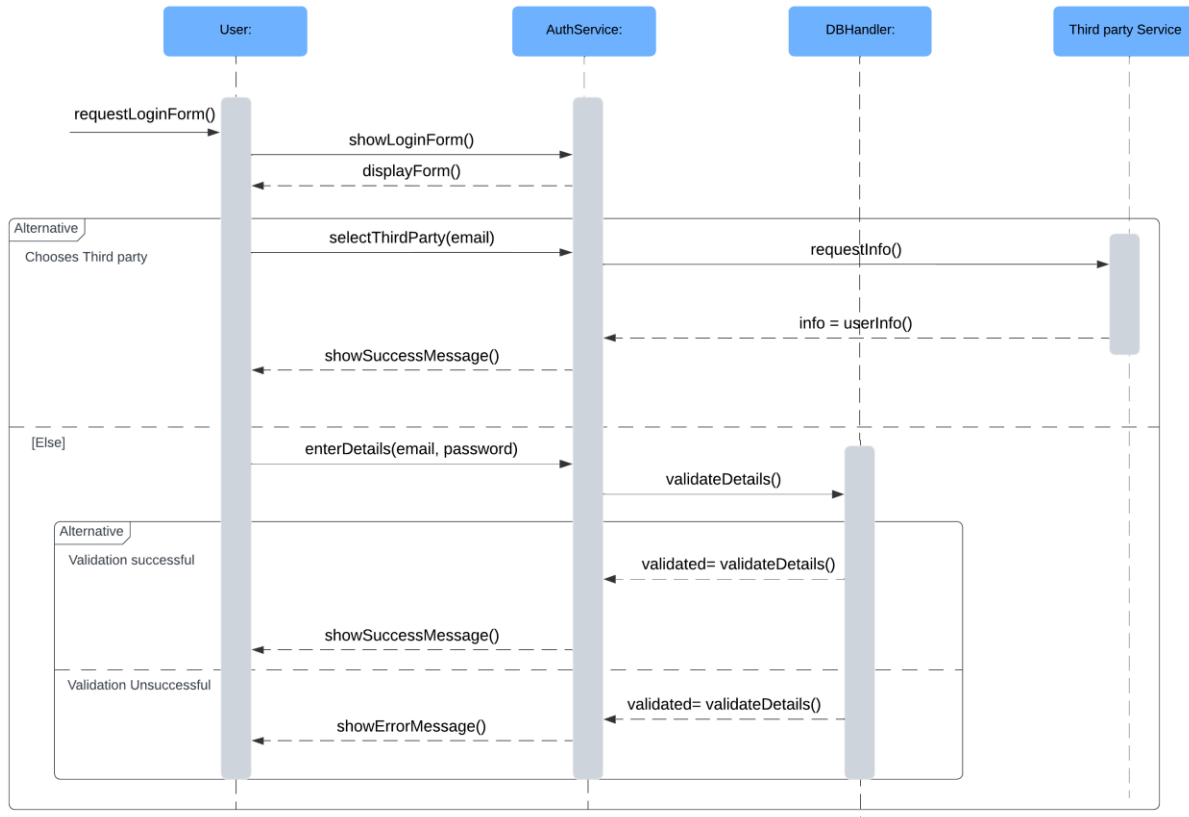


Figure 15: Login Sequence System

3.2.3.3 Reset Password

This sequence diagram outlines the process for resetting a user's password. The user initiates the process by requesting the reset password form and entering their email. The system validates the email and, if successful, sends a PIN to the user via email. The user then enters the PIN, which is validated by the system. If the PIN is correct, the system allows the user to enter a new password. The new password is saved, and the user is shown a success message. If the validation of the email, PIN, or new password fails at any point, the system returns an error message, prompting the user to retry.

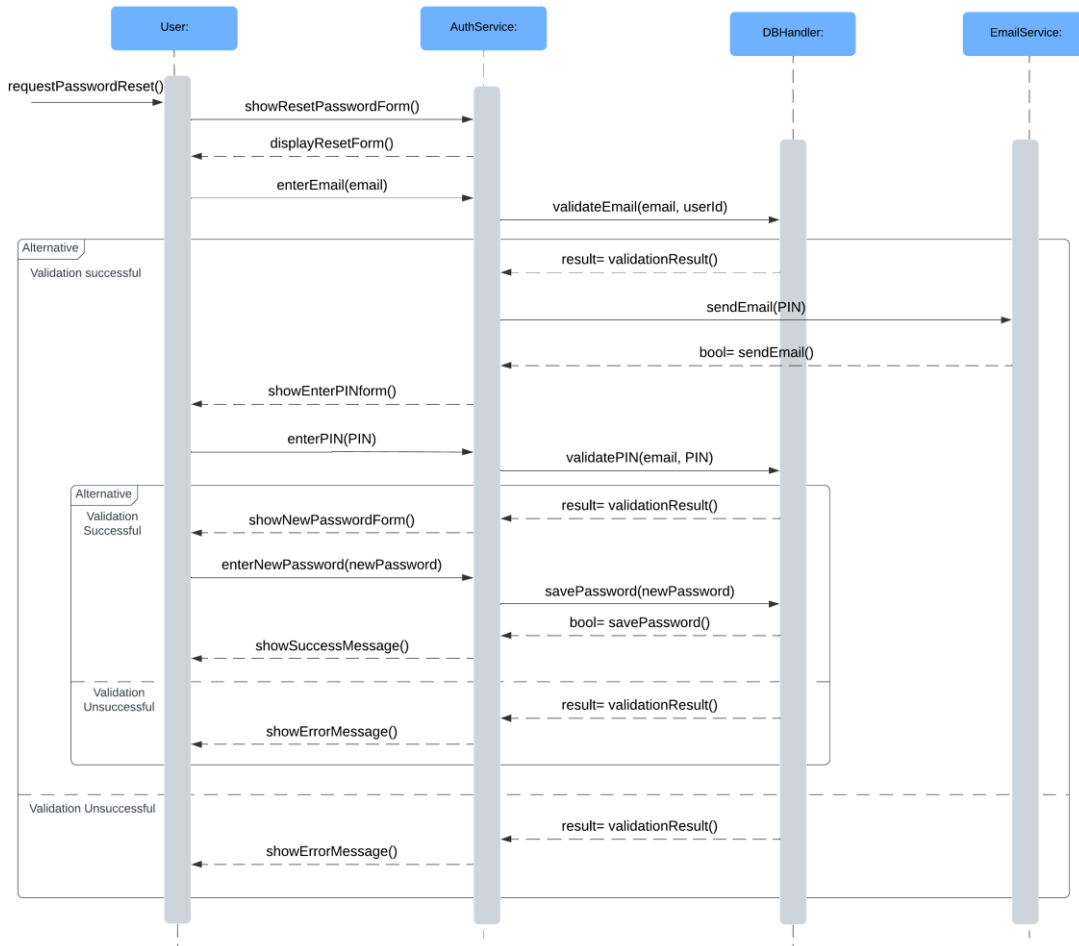


Figure 16: Reset Password Sequence Diagram

3.2.3.4 Select a game

In this sequence diagram, the user starts by selecting the learning section. The system retrieves the available topics and displays them to the user. The user then selects a game from the available topics. The system fetches the corresponding game data from the database and loads the game environment, allowing the user to begin playing the selected game.

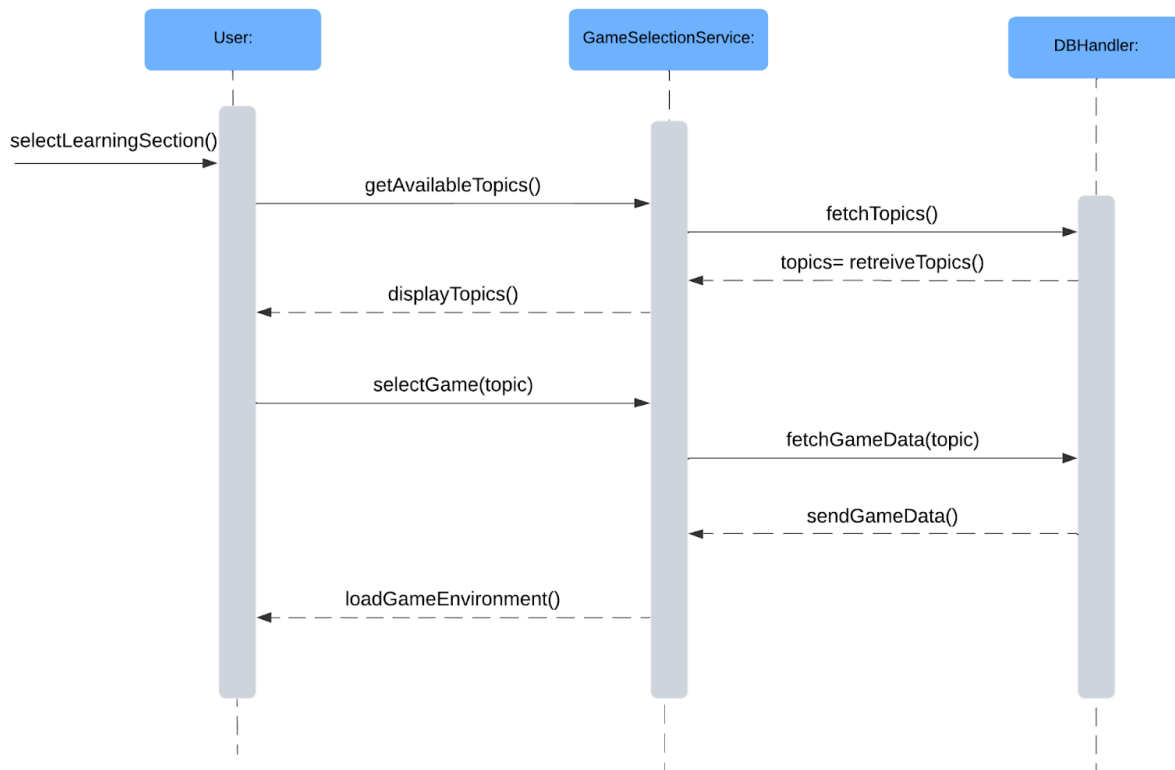


Figure 17: Select Game Sequence Diagram

3.2.3.5 Play a game

This sequence diagram demonstrates the process of playing a game. The user begins by starting the game, after which the system displays instructions and retrieves the necessary game information. The user makes a move, and the system checks if the move is valid. If the move is correct, the system shows a success message; if the move is incorrect, an error message is displayed. This process repeats until the game ends. Once the game is finished, the system saves the user's progress.

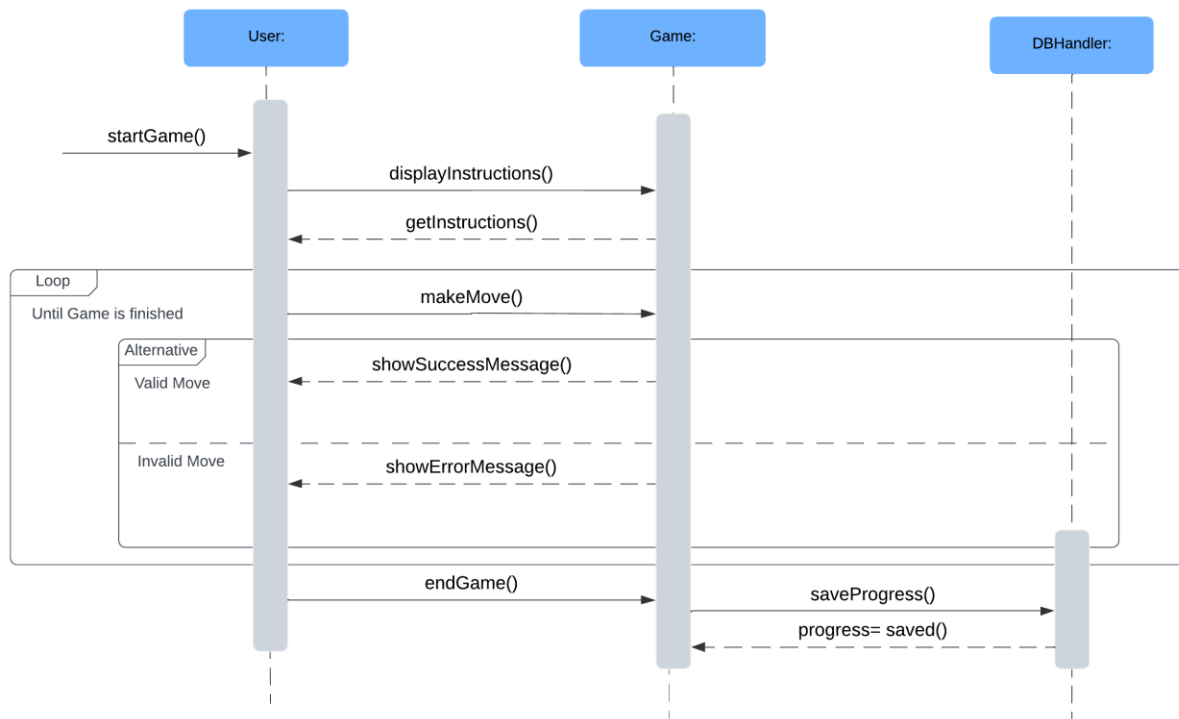


Figure 18: Select a Game Sequence Diagram

3.2.4 Class Diagram:

This class diagram provides a comprehensive view of the system's architecture, illustrating the key components and their relationships. The main components are broken down into several classes, each representing distinct entities in the system.

1. **User Interface Class:** This serves as the base interface for users, containing attributes like `userID`, `userName`, `email`, and `password`, along with basic functions like `register()`, `login()`, `logout()`, and `updateProfile()`. It is extended by the Admin and Learner classes.
2. **Admin Class:** Inherits from the User interface and manages users through the function `manageUsers()`.
3. **Learner Class:** Inherits from the User interface and has access to additional functionalities such as viewing rewards, feedback, progress, and taking lessons or quizzes. It maintains relationships with lessons, quizzes, and games.
4. **Lesson Class:** Stores details of each lesson, such as `lessonID`, `title`, and associated games. It has functions like `startLesson()` and `completeLesson()`. Lessons are part of a Learning Path.

5. **Reward Class:** Represents the rewards that users can receive. It contains details like rewardID, title, description, and points. The function assignReward() allows assigning rewards to learners.
6. **Game Class:** Contains attributes like gameId, score, and lives. It manages the gameplay through methods like playGame(), checkMove(), updateScore(), and endGame().
7. **Quiz Class:** Contains the list of questions, lives, and methods such as startQuiz(), submitQuiz(), and calculateScore(). Each quiz has multiple questions.
8. **Question Class:** Represents individual questions in quizzes, storing the question text, options, and correct answers.
9. **Learning Path Class:** Organizes lessons and assessments, providing functions like generatePath() and updatePath(). It helps learners follow a structured learning process.
10. **Feedback Class:** Stores feedback related to lessons or quizzes, with methods to generate feedback for users.
11. **Assessment Class:** Holds assessment details like assessmentID, associated questions, and results. It has functions like startAssessment(), submitAssessment(), and calculateScore().
12. **Progress Class:** Tracks a learner's progress, including completed lessons, total score, and remaining lives, with methods to update and view progress.
13. **Code Visualization Class:** Handles the visualization of code, helping learners understand the structure and logic of their solutions. It includes functions to generate and update visualizations of code.
14. **Score Class:** Manages the scores associated with quizzes and assessments, with methods like calculateScore() and updateScore().

Each of these classes interacts with others through associations, including one-to-one, one-to-many, and many-to-many relationships, creating a robust framework for the system's functionality. The diagram highlights the modular structure of the system, ensuring that each part operates independently but contributes to the overall user learning experience.

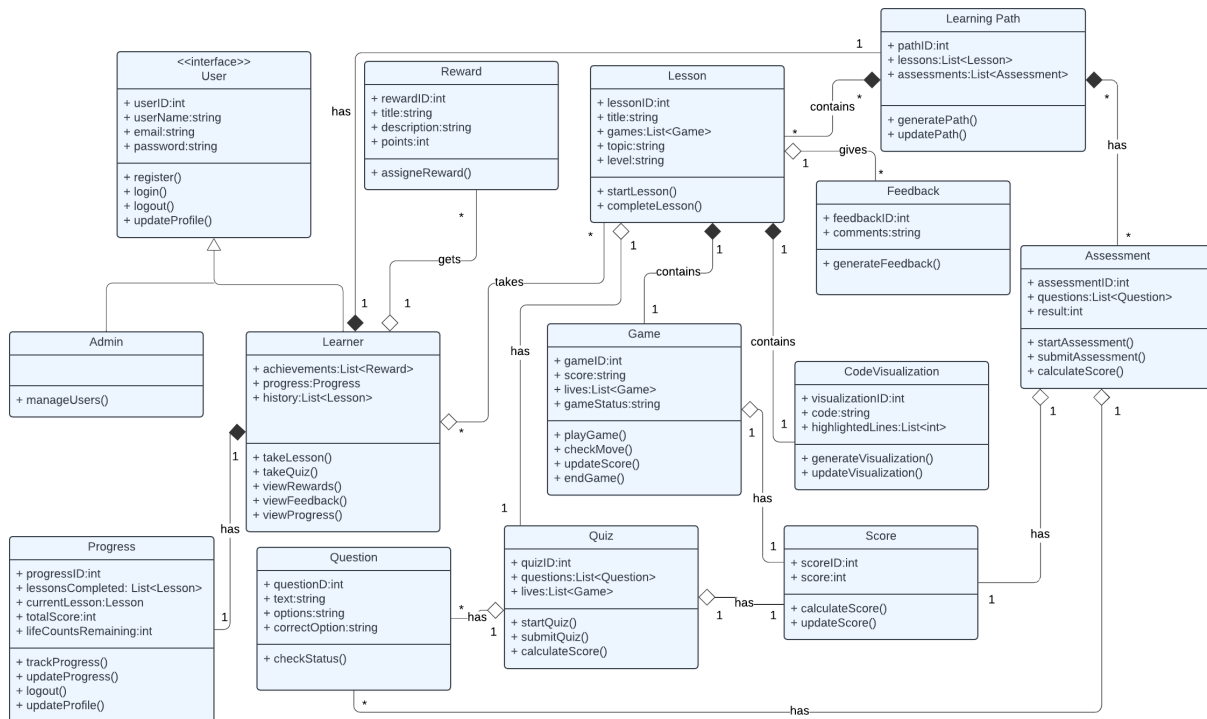


Figure 19: Class Diagram

3.3 Entity Relationship Diagram

The provided ERD represents the structure of your system, showing the relationships between key entities such as Admin, Learner, Progress, Reward, Lesson, Game, Quiz, Score, and others. Each entity includes relevant attributes, such as learner ID, game ID, question ID, and reward ID, which act as primary keys. The diagram outlines how learners can access lessons, take quizzes, and receive rewards for their progress. Admins manage users, and feedback is given based on game performance. Code visualizations and learning paths are integrated into the system, along with assessments that are connected to scores. This ERD helps clarify how data flows between different entities and how relationships such as one-to-many and many-to-many are structured, essential for implementing the system's database design.

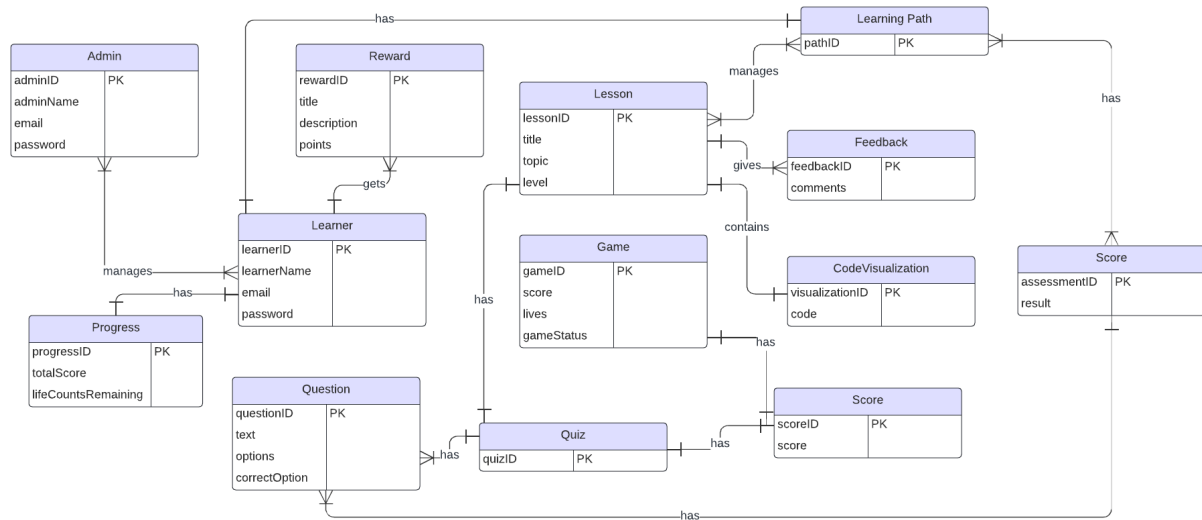


Figure 20: ERD

References

- [1] Mtaho, A. B., & Mselle, L. J. (2024). Difficulties in learning the data structures course: Literature review. *The Journal of Informatics*, 4(1), 26-55.